

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 10

Medienverarbeitung

Medienverarbeitung bezeichnet den Prozess des Erstellens und Einlesens von Medienformaten abseits von HTML. Im Konkreten werden hier mehrere Dinge angesprochen: das Lesen und Schreiben von XML und JSON, die Erstellung von PDF und das Laden und Konvertieren von Grafiken.

10.1 XML-Verarbeitung

XML steht für *Extensible Markup Language*, eine vom W3C standardisierte Markup-Sprache. Eine detaillierte Ausführung zu den Grundlagen von XML findet sich in den Unterlagen zu Vorlesung 7 aus dem Wintersemester.

Um XML maschinell zu verarbeiten, gibt es eine Reihe von Werkzeugen, die eingesetzt werden können: Parser, Transformatoren und Renderer. Für diese Lehrveranstaltung relevant sind die XML-Parser.

10.1.1 Der Parser

Die Aufgabe eines *XML-Parsers* ist es, ein XML-Dokument einzulesen, und die darin enthaltenen Informationen programmiertechnisch aufzubereiten. Der Parser muss dafür die Struktur des Dokuments, das er abarbeiten soll, genau kennen. D. h., es gibt nicht *den* XML-Parser, sondern es gibt einen spezifischen Parser für jede XML-Anwendung. Wird eine eigene XML-Anwendung definiert, so muss meist auch ein eigener Parser dazu geschrieben werden, um Daten auslesen zu können. Dieser Parser weiß somit genau, welche Elemente und Attribute im XML-Dokument zu erwarten sind. Die Unbekannten sind allerdings die Daten dahinter, also die Werte der einzelnen Elemente und Attribute. Diese werden mit Hilfe des Parsers ausgelesen.

Die Arbeit mit einem Parser lässt sich in drei große Schritte unterteilen:

1. **Initialisieren des Parsers:** Hierbei wird der Parser im Programm angelegt und alle notwendigen Einstellungen werden vorgenommen.
2. **Aufruf des Parsers:** Dem Parser wird das XML-Dokument, das ausgelesen werden soll, übergeben und der Parser beginnt, es abzuarbeiten.
3. **Verarbeiten der Ergebnisse:** Die vom Parser gelieferten Daten werden verarbeitet.

Ein XML-Parser kann bei der Abarbeitung des Dokuments validieren, d. h. die XML-Grammatik überprüfen und ggf. auch korrigieren (implementierungsabhängig). Tut er dies, so ist er in der Lage, auftretende Fehler zu benennen und anzuzeigen.

Arten von XML-Parsern

Um ein XML-Dokument zu verarbeiten, gibt es in sehr vielen Programmiersprachen (z. B. C++, Java, PHP etc.) XML-Bibliotheken, die XML-Parser beinhalten. Diese lassen sich in drei große Kategorien einteilen:

Eventbasierte Parser: Eventbasierte Parser, auch Push- oder SAX-Parser (Simple API for XML) genannt, repräsentieren das XML-Dokument als sequentiellen Datenstrom. Das XML-Dokument wird Schritt für Schritt durchlaufen und an wichtigen Stellen (z. B. Beginn eines Tags, Beginn von Inhalt, Ende eines Tags etc.) wird automatisch eine von dem*der Entwickler*in gewählte Funktion (genannt Callback-Funktion) aufgerufen, die die Verarbeitung übernimmt. Das Antreffen einer solchen wichtigen Position im Dokument wird als Event bezeichnet, daher auch der Name. Eventbasierte Parser sind sehr speichereffizient, sind jedoch aufwändiger für Entwickler*innen, da diese nur die Informationen bekommen, wo der Parser gerade arbeitet (z. B. am Beginn eines Elements). Entwickler*innen müssen sich selber im Code darum kümmern, alle weiteren Informationen (wie Name des Tags etc.) zu erhalten und sich ebenfalls merken, welche Teile vom Dokument bereits verarbeitet wurden.

Eventbasierte Parser sind gut geeignet für einfache Operationen, wenn etwa nur bestimmte Elemente eines Dokuments ausgelesen werden sollen. Da Schritt für Schritt gelesen wird, sind sie sehr gut für große XML-Dokumente zu gebrauchen. Die Struktur eines XML-Dokuments kann damit jedoch nicht verändert und neue Dokumente können nicht erzeugt werden, da der Parser nur zum Lesen geeignet ist. Weiters können Inhalte nicht validiert werden.

PHP bietet mit dem Modul *XML Parser*¹ einen eventbasierten (SAX) Parser an. Dieser ist der am längsten verfügbare, er existiert bereits seit PHP 4 und ist von der API am umständlichsten zu verwenden. Er wird daher nicht näher erwähnt.

Pull-Parser: Ein Pull-Parser behandelt ein XML-Dokument als eine Serie von aufeinanderfolgenden Elementen und verwendet einen so genannten Iterator, um diese der Reihe nach abzuarbeiten. Dabei wird das Durchlaufen der verschiedenen Elemente von dem*der Entwickler*in bestimmt. Während bei SAX-Parsing die Informationen vom Parser permanent beim Auftreten eines Events geschickt werden (ob benötigt oder nicht), entscheiden Entwickler*innen beim Pull-Parsing, ob sie an einer bestimmten Stelle im Dokument Daten verarbeiten möchten. D. h. diese werden nur angefordert, wenn es nötig ist, wovon sich auch der Name Pull-Parser ableitet.

In PHP steht mit dem Modul *XMLReader*² ein voll implementierter Pull-Parser zur Verfügung.

¹<https://www.php.net/manual/de/book.xml.php>

²<https://www.php.net/manual/de/book.xmlreader.php>

Objektorientierte Parser: Ein objektorientierter Parser stellt das XML-Dokument als Objektstruktur dar und bietet wahlfreien Zugriff auf die einzelnen Elemente – in diesem Zusammenhang meist Knoten genannt. Ein Knoten ist ein Element mit Inhalt und ggf. Attributen. Objektorientierte Parser sind sehr praktisch, weil auf Elemente sehr einfach zugegriffen werden kann, bei großen Dateien benötigen sie jedoch viel Speicher, da sie ja die komplette Struktur auf einmal laden müssen.

In PHP stehen standardmäßig zwei Parser zur Verfügung, die nach diesem Modell arbeiten: *DOM*³ und *SimpleXML*⁴.

10.1.2 XML einlesen in PHP

Im Folgenden werden zwei der drei in PHP verfügbaren Parser-Arten – der Pull-Parser und ein objektorientierter Ansatz – anhand eines Beispiels besprochen. Als Dokument dient das Rezept für Tirolerknödel, welches bereits im Wintersemester vorgestellt wurde. Dieses Rezept soll geparsed und sein Inhalt lesbar ausgegeben werden.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <rezept quelle="https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386">
3   <gericht>Der klassische Tiroler Knödel</gericht>
4   <zutaten>
5     <zutat>
6       <ingredienz>Semmelwürfel</ingredienz>
7       <menge>350</menge>
8       <einheit>g</einheit>
9     </zutat>
10    <zutat>
11      <ingredienz>Milch</ingredienz>
12      <menge>200</menge>
13      <einheit>ml</einheit>
14    </zutat>
15    <zutat>
16      <ingredienz>Eier</ingredienz>
17      <menge>4</menge>
18      <einheit>Stück</einheit>
19    </zutat>
20    <zutat>
21      <ingredienz>Speck</ingredienz>
22      <menge>100</menge>
23      <einheit>g</einheit>
24    </zutat>
25    <zutat>
26      <ingredienz>Bergsteigerwurst</ingredienz>
27      <menge>100</menge>
28      <einheit>g</einheit>
29    </zutat>
30    <zutat>
31      <ingredienz>Butter</ingredienz>
32      <menge>20</menge>
33      <einheit>g</einheit>
34    </zutat>
35    <zutat>
```

³<https://www.php.net/manual/de/book.dom.php>

⁴<https://www.php.net/manual/de/book.simplexml.php>

```

36     <ingredienz>Zwiebel</ingredienz>
37     <menge>1</menge>
38     <einheit>Stück</einheit>
39 </zutat>
40 <zutat>
41     <ingredienz>Petersilie</ingredienz>
42     <menge>10</menge>
43     <einheit>g</einheit>
44 </zutat>
45 <zutat>
46     <ingredienz>Salz</ingredienz>
47     <menge>1</menge>
48     <einheit>Prise</einheit>
49 </zutat>
50 <zutat>
51     <ingredienz>Muskatnuss</ingredienz>
52     <menge>1</menge>
53     <einheit>Prise</einheit>
54 </zutat>
55 <zutat>
56     <ingredienz>Mehl</ingredienz>
57     <menge>80</menge>
58     <einheit>g</einheit>
59 </zutat>
60 <zutat>
61     <ingredienz>Schnittlauch</ingredienz>
62     <menge>10</menge>
63     <einheit>g</einheit>
64 </zutat>
65 </zutaten>
66 <zubereitung>
67     <schritt>Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen
        lassen.</schritt>
68     <schritt>Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in
        zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten
        und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen
69     <schritt>Mit befeuchteten Händen daraus Knödel formen.</schritt>
70     <schritt>Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat
        der Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die
        Knödelmasse kneten.</schritt>
71     <schritt>Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.
        </schritt>
72     <schritt>Tiroler Knödel mit Schnittlauch bestreut servieren.</schritt>
73 </zubereitung>
74 </rezept>

```

Pull-Parser: XMLReader

Pull-Parser arbeiten das Dokument sequentiell ein. Man spricht hier auch von cursorbasiertem Einlesen, da quasi ein Cursor durch das Dokument „geschoben“ wird). Wann immer der Parser auf einen der unterschiedlichen Teile des Dokuments (Tags, Inhalt, Attribute etc.) stößt, gibt es die Möglichkeit, diese zu verarbeiten. Dabei wird das Knoten-Konzept von den objektorientierten DOM-Parsern übernommen: jeder Eintrag im XML-Baum wird als Knoten angesehen.

In einer `while`-Schleife läuft der Parser über das Dokument, dabei wird der Cursor von Knoten zu Knoten weiterbewegt. Nun muss festgestellt werden, um welche Art von Knoten (Element, Text, schließendes Element) es sich handelt. Danach können die Eigenschaften des Knoten abgefragt und der Inhalt verarbeitet werden.

In PHP wird ein Pull-Parser durch die Erweiterung *XMLReader* zur Verfügung gestellt. Sie ist seit PHP 5.1 Teil der Standarddistribution und arbeitet vollkommen objektorientiert.

Im Folgenden wird das Knödelrezept mit dem Pull-Parser verarbeitet und ausgegeben. Es wird dabei auf die einzelnen Teile des Parse-Vorgangs genauer eingegangen. Der ganze Code wird in der Klasse `MyPullParser` verpackt.

Parser initialisieren: Zum Speichern der auszulesenden Werte werden zunächst Eigenschaften in `MyPullParser` mit *Asymmetric Visibility* (`private(set)`) angelegt. Dies erlaubt es, Eigenschaften von außen lesend, aber nur von innen schreibend zuzugreifen (Getter-Methoden entfallen). Für die Initialisierung des Parsers muss nur ein Objekt der Klasse `XMLReader` angelegt werden. Die Klasse `MyPullParser` sieht im Grundgerüst wie folgt aus:

```
1 namespace XMLExample;
2
3 use XMLReader;
4
5 class MyPullParser
6 {
7     private XMLReader $parser;
8
9     private(set) string $source = "";
10    private(set) string $dish = "";
11    private(set) array $ingredients = [];
12    private(set) array $steps = [];
13
14    public function __construct()
15    {
16        $this->parser = new XMLReader();
17    }
18
19    // ...
20 }
```

Parsen starten: Das Starten des Parse-Vorgangs beginnt mit dem Öffnen der einzulesenden Datei mit Hilfe der Methode `open($file)`⁵. Danach wird das Einlesen durch Aufruf der Methode `read()`⁶ am Parser-Objekt begonnen. Diese Methode muss in einer `while`-Schleife aufgerufen werden, denn sie liefert solange `true`, solange Inhalt zum Einlesen vorhanden ist, und bricht die Schleife dann am Ende des XML-Dokuments durch das Zurückgeben von `false` ab.

```
1 public function parse(string $file): void
2 {
3     $this->parser->open($file);
```

⁵<https://www.php.net/manual/de/xmlreader.open.php>

⁶<https://www.php.net/manual/de/xmlreader.read.php>

```

4
5 while ($this->parser->read()) {
6     // Verarbeitung...
7 }
8 }

```

Inhalte verarbeiten: Die Methode `read()` läuft nun sequentiell durch das XML-Dokument und stoppt bei jedem Knoten. Als Knoten wird – wie auch schon vom DOM bekannt – ein jeder Bestandteil des Dokuments angesehen. Knoten haben wiederum verschiedene Typen und werden durch Konstanten in der `XMLReader`-Klasse bezeichnet. Eine Auflistung findet sich im PHP Handbuch⁷, die drei wichtigsten lauten:

- `XMLReader::ELEMENT`: Öffnendes Element.
- `XMLReader::END_ELEMENT`: Schließendes Element.
- `XMLReader::TEXT`: Text zwischen Tags.

Um nun festzustellen, bei welcher Art von Knoten der Cursor stehen geblieben ist, kann die Eigenschaft `nodeType` (z. B. mittels einer `switch`-Anweisung) gegen diese Konstanten überprüft werden. Anschließend können im jeweiligen `case`-Teil die Operationen für öffnende Elemente, schließende Elemente und Text durchgeführt werden. Im aktuellen Fall genügt es jedoch, mittels `if`-Bedingung auf ein öffnendes Element abzufragen, da alle Operationen dort erledigt werden können.

Handelt es sich nun um ein öffnendes Element, erfolgt das Auslesen der Inhalte. Dazu wird in einer `switch`-Anweisung der Name des aktuellen Element-Knotens als Entscheidungskriterium herangezogen. Handelt es sich um das `<rezept>`-Element, wird das Attribut `quelle` abgefragt und in der Eigenschaft `$source` gespeichert. Beim Element `<gericht>` wird die Methode `readString()`, die den Textinhalt des Knotens zurückgibt, aufgerufen und das Ergebnis in die Eigenschaft `$dish` gespeichert. Die Elemente `<ingredienz>`, `<menge>`, `<einheit>` und `<schritt>` werden auf dieselbe Art und Weise in die Arrays `$ingredients` bzw. `$steps` ausgelesen. Somit ist die Verarbeitung abgeschlossen.

Anmerkung: Anstatt mit `readString()` beim öffnenden Element den Inhalt auszulesen, hätte auch auf einen `TEXT`-Knoten überprüft und dessen Wert zurückgegeben werden können. Hierbei muss aber wiederum sichergestellt werden, zu welchem Element dieser Text gehört (z. B. durch eine Variable `$currentElement`, die sich das öffnende Element merkt).

```

1 while ($this->parser->read()) {
2     if ($this->parser->nodeType === XMLReader::ELEMENT) {
3         switch ($this->parser->name) {
4             case "rezept":
5                 $this->source = $this->parser->getAttribute("quelle") ? "";
6                 break;
7             case "gericht":
8                 $this->dish = trim($this->parser->readString());
9                 break;
10            case "zutat":
11                $this->ingredients[] = [];
12                break;

```

⁷<https://www.php.net/manual/de/class.xmlreader.php>

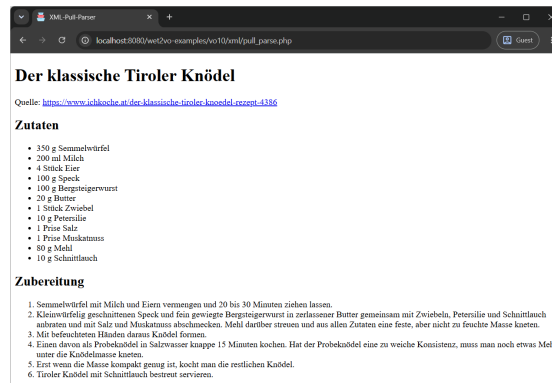


Abbildung 10.1: Im Browser wird der Inhalt des XML-Dokuments strukturiert angezeigt.

```

13     case "ingredienz":
14     case "menge":
15     case "einheit":
16         $lastIndex = array_key_last($this->ingredients);
17         $this->ingredients[$lastIndex][$this->parser->name] = trim($this->parser->
    readString());
18         break;
19     case "schritt":
20         $this->steps[] = trim($this->parser->readString());
21         break;
22     }
23 }
24 }
```

Ablauf starten: Um den Ablauf zu starten, muss ein Objekt der Klasse `MyPullParser` angelegt und darauf die Methode `parse()` mit der zu parsenden XML-Datei aufgerufen werden. Danach kann über die Eigenschaften, die öffentlich lesbar sind, das Ergebnis abgefragt und entsprechend ausgegeben werden. Das Ergebnis im Browser ist in Abbildung 10.1 ersichtlich.

```

1 $xmlParser = new MyPullParser();
2 $xmlParser->parse("rezept.xml");
```

Parser zerstören: Um den Parser zu zerstören, bietet sich der Destruktor an. Dazu nötig ist lediglich der Aufruf der Methode `close()`⁸, welche sich auch selbst darum kümmert, den Parser zu stoppen, sollte dieser gerade mit dem Verarbeiten eines Dokuments beschäftigt sein.

```

1 public function __destruct()
2 {
3     $this->parser->close();
4 }
```

⁸<https://www.php.net/manual/de/xmlreader.close.php>

Objektorientierter Parser: DOM

Neben dem soeben vorgestellten Pull-Parser (und dem hier nicht erwähnten eventbasierten Parser) existiert das dritte Modell der *objektorientierten Parser*. Diese Parser arbeiten mit dem Document Object Model (DOM) und repräsentieren die XML-Struktur als DOM-Baum. Da der Parser immer die gesamte Struktur des Dokuments kennt (da er den Baum vollständig aufbauen muss), ist es einfach, bestimmte Teile des Dokuments selektiv auszulesen. Dies bedeutet jedoch, dass das gesamte Dokument im Speicher gehalten werden muss, was bei großen XML-Dokumenten zum Problem werden kann.

PHP bietet seit Version 5 zwei verschiedene objektorientierte Parser an. Zunächst *SimpleXML*⁹, eine einfache Erweiterung, die XML-Objekte als eine Ansammlung von verschachtelten PHP-Objekten repräsentiert. Weiters auch *DOM*, das den W3C DOM Living Standard implementiert und somit einen Zugriff auf das Dokument mit den bekannten DOM-Methoden ermöglicht. Letzteres wird hier vorgestellt.

Der *DOM-Parser* wurde mit PHP 8.4 grundlegend überarbeitet und entspricht nun vollständig dem DOM Living Standard. Er bildet das gesamte XML-Dokument in einer Baumstruktur ab. Über die aus der JavaScript DOM API bekannten Methoden lassen sich leicht bestimmte Knoten (Elemente) finden.

Dies sei am Beispiel des Knödelrezepts demonstriert. In der Klasse *MyDOMParser* werden die folgenden DOM-Parser Funktionen abgebildet.

Parser initialisieren und Parsen starten: Beim DOM-Parser ist der Initialisierungsaufwand gering. Soll ein XML-Dokument eingelesen werden, wird ein Objekt der Klasse *Dom\XMLDocument*¹⁰ mit Hilfe des statischen Konstruktors *createFromFile()* angelegt. Darum ist es auch flexibler, dies gleich in der Methode *parse()* und nicht im Konstruktor zu erledigen. Dies startet den Parser, dieser läuft über das gesamte Dokument und bildet dessen Struktur als Baum im Objekt *\$dom* ab. Danach ist das DOM-Objekt befüllt und kann für Abfragen verwendet werden.

```

1 class MyDOMParser
2 {
3     private(set) string $source = "";
4     private(set) string $dish = "";
5     private(set) array $ingredients = [];
6     private(set) array $steps = [];
7
8     public function parse($file)
9     {
10         $this->source = "";
11         $this->dish = "";
12         $this->ingredients = [];
13         $this->steps = [];
14
15         $dom = XMLDocument::createFromFile($file);
16
17         // Verarbeitung
18     }
19 }
```

⁹<https://www.php.net/manual/de/book.simplexml.php>

¹⁰<https://www.php.net/manual/de/class.dom-xml-document.php>

Inhalte verarbeiten: Die Stärke des DOM-Ansatzes liegt darin, direkte Abfragen machen zu können. Dabei muss die Struktur des Dokuments bekannt sein. Analog zu JavaScript stehen die üblichen Funktionen und Eigenschaften zur Verfügung:

- `documentElement`¹¹: Enthält den Wurzelknoten.
- `querySelector($selectors)`¹²: Gibt das erste Element zurück, das dem übergebenen CSS-Selektor entspricht.
- `querySelectorAll($selectors)`¹³: Gibt eine `NodeList` mit allen Elementen zurück, die dem übergebenen CSS-Selektor entsprechen.
- `getElementById($id)`¹⁴: Gibt das Element mit der gegebenen ID zurück.
- ...

Wurden die gewünschten Elemente gefunden, gibt es wiederum bekannte Eigenschaften und Methoden, um Informationen daraus zu erhalten:

- `nodeName`: Der Name eines Knotens.
- `nodeType`: Der Typ eines Knotens (Element, Text etc.).
- `textContent`: Der Textinhalt eines Knotens.
- `firstChild`: Der erste Kindknoten.
- `hasChildNodes()`: Hat der Knoten weitere Kindknoten?
- ...

Diese Methoden und Eigenschaften können nun verwendet werden, um entsprechende Informationen aus dem Dokument abzufragen. Das Wurzelelement `<rezept>` wird über die Eigenschaft `documentElement` abgefragt. `getAttribute($name)`¹⁵ wird verwendet, um das dazugehörige `quelle`-Attribut abzufragen.

Um das `<gericht>`-Element zu erhalten, wird mit `querySelector("gericht")` das erste `<gericht>`-Element abgefragt und dessen Wert mit `textContent` gespeichert.

Ähnlich werden die einzelnen Zutaten verarbeitet. Zunächst werden alle `<zutat>`-Elemente mit `querySelectorAll()` abgefragt und dann mit einer `foreach`-Schleife durchlaufen. Danach werden für jedes Element dessen Kind-Knoten abgefragt. Durch die Typprüfung `$child instanceof Element` wird sichergestellt, dass es sich um einen echten Element-Knoten und nicht um unnötige Text-Knoten (wie Zeilenumbrüche) handelt. Ist dies der Fall, wird der Wert in `$ingredients` gespeichert.

Auch beim Auslesen der Zubereitungsschritte läuft es ähnlich: Zunächst werden alle `<schritt>`-Elemente abgefragt, danach werden in einer Schleife die Inhalte in `$steps` gespeichert.

```
1 $this->source = $dom->documentElement->getAttribute("quelle") ?? "";
2
3 $this->dish = trim($dom->querySelector("gericht")->textContent);
4
5 foreach ($dom->querySelectorAll("zutat") as $ingredient) {
6     $this->ingredients[] = [];
7     foreach ($ingredient->childNodes as $child) {
```

¹¹<https://www.php.net/manual/de/class.domdocument.php#domdocument.props.documentelement>

¹²<https://www.php.net/manual/de/dom-parentnode.querySelector.php>

¹³<https://www.php.net/manual/de/dom-parentnode.querySelectorAll.php>

¹⁴<https://www.php.net/manual/de/domdocument.getelementbyid.php>

¹⁵<https://www.php.net/manual/de/domelement.getattribute.php>

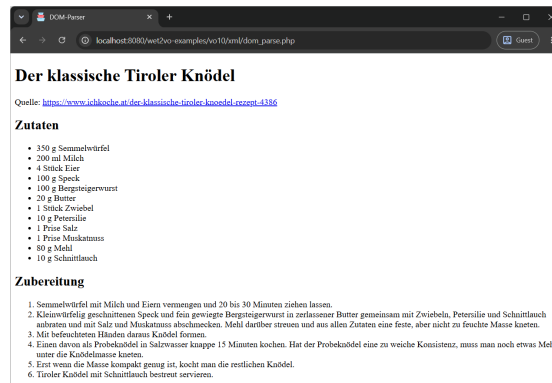


Abbildung 10.2: Im Browser wird das XML-Dokument strukturiert ausgegeben. Das Ergebnis identisch zum vorhergehenden Ansatz.

```

8     if ($child instanceof Element) {
9         $lastIndex = array_key_last($this->ingredients);
10        $this->ingredients[$lastIndex][$child->tagName] = trim($child->textContent);
11    }
12 }
13 }
14
15 foreach ($dom->querySelectorAll("schritt") as $step) {
16     $this->steps[] = trim($step->textContent);
17 }

```

Ablauf starten und Ausgabe: Um den Ablauf zu starten, wird ein Objekt der Klasse `MyDOMParser` erzeugt und darauf die `parse()`-Methode aufgerufen:

```

1 $xmlParser = new MyDOMParser();
2 $xmlParser->parse("rezept.xml");

```

Die Ausgabe des DOM-Parse-Vorgangs ist in Abbildung 10.2 ersichtlich.

10.1.3 XML erzeugen in PHP

Das *Erzeugen von XML* ist der umgekehrte Prozess zum Parsen. Aus einem Set von Daten wird eine strukturierte XML-Datei erstellt. Dabei kann die XML-Datei entweder im Browser ausgegeben oder in einer Datei gespeichert werden. Als Beispiele sollen TV-Shows in XML-Form abgespeichert werden.

Die Sendungen seien im folgenden Array gegeben:

```

1 $this->shows = [
2     [
3         "name" => "The Mandalorian",
4         "service" => "Disney+",
5         "resolution" => "2160p",
6         "duration" => "40",
7     ],
8     [
9         "name" => "Rick and Morty",

```

```
10     "service" => "Netflix",
11     "resolution" => "1080p",
12     "duration" => "20",
13 ],
14 ];
```

Da XML ein Textformat ist, kann es auch völlig *ohne weitere Tools* erzeugt werden. Für simple Strukturen ist dies oft auch völlig ausreichend, wenngleich es nicht die eleganteste Art und Weise ist. In diesem Abschnitt werden jedoch die Erweiterungen *XMLWriter* und *DOM* vorgestellt, die diese Aufgabe effizienter erledigen.

XMLWriter

Eine komfortable Methode des XML-Schreibens ist die Verwendung der Erweiterung *XMLWriter*. Sie ist das Gegenstück zum *XMLReader* und auch seit PHP 5.1 standardmäßig inkludiert. XMLWriter arbeitet ebenfalls sequentiell, d. h. die einzelnen Teile der XML-Datei müssen in der richtigen Reihenfolge definiert werden, man bezeichnet sie daher auch als Stream-Writer. Die Stärke dieser Erweiterung liegt vor allem im Neuerstellen von XML-Inhalten, weniger in der Veränderung von bestehenden Dateien. Dies ist durch die sequentielle Abarbeitung bedingt.

XMLWriter kann sowohl funktional als auch objektorientiert verwendet werden. Die objektorientierte Variante ist jedoch wesentlich eleganter und wird daher in weiterer Folge verwendet. Die folgenden Programmteile gehören zur Klasse *MyStreamWriter*.

Writer initialisieren: Zunächst muss ein neues *XMLWriter*-Objekt erzeugt werden. Diesem Objekt wird dann mitgeteilt, ob die XML-Datei ausgegeben, oder in eine Datei geschrieben werden soll. Für die Ausgabe am Bildschirm wird die XML-Datei in den Speicher geschrieben, daher wird die Methode `openMemory()`¹⁶ benötigt. Für die Ausgabe in eine Datei kommt `openUri($uri)`¹⁷ zum Einsatz. Damit die Elemente korrekt und gut leserlich eingerückt sind, kann über die Funktion `setIndent($indent)`¹⁸ diese Einrückung aktiviert werden.

Im Beispiel wird lediglich das Objekt im Konstruktor angelegt. Das Öffnen der Datei sowie das Setzen der Einrückung findet bereits in der Methode zum Schreiben statt.

```
1 class MyStreamWriter
2 {
3     private XMLWriter $writer;
4     private array $shows;
5
6     public function __construct(array $shows)
7     {
8         $this->writer = new XMLWriter();
9         $this->shows = $shows;
10    }
11
12    public function generateXML(string $file): void
13    {
14        $this->writer->openUri($file);
```

¹⁶<https://www.php.net/manual/de/xmlwriter.openmemory.php>

¹⁷<https://www.php.net/manual/de/xmlwriter.openuri.php>

¹⁸<https://www.php.net/manual/de/xmlwriter.setindent.php>

```

15     $this->writer->setIndent(true);
16
17     // Verarbeitung
18 }
19 }

```

XML erzeugen: Nachdem der Writer konfiguriert wurde, werden der Reihe nach die Bestandteile der XML-Datei erzeugt. Begonnen wird mit der XML-Deklaration. Diese wird durch den Aufruf von `startDocument($version, $encoding)`¹⁹ erzeugt. Gleichzeitig wird damit auch das Dokument begonnen. Als Version sollte 1.0, als Encoding UTF-8 angegeben werden.

In der Regel folgt im nächsten Schritt eine DTD, die unter Zuhilfenahme der Methode `writeDtd($name [, $publicId, $systemId [, $content]])`²⁰ geschrieben werden kann. Im aktuellen Beispiel wird darauf verzichtet.

Es soll nun das Wurzelement `<shows>` erzeugt werden. Da es zunächst nur geöffnet und erst ganz am Schluss wieder geschlossen wird, kommt `startElement($name)`²¹ zum Einsatz. Die Methode erzeugt ein öffnendes Element mit einem gegebenen Namen. Hier wäre der Zeitpunkt, ein XML-Schema anzugeben. Dazu würden die entsprechenden Attribute wie `xmlns:xsi` mit der Methode `writeAttribute($name, $value)`²² hinzugefügt.

Danach sollen innerhalb eines `<show>`-Elements die Details der einzelnen Serien erzeugt werden. Es wird wiederum eine `foreach`-Schleife verwendet, um über das Array mit den Shows zu iterieren. Mittels `startElement()` wird das `<show>`-Element geöffnet. Dann kommt die innere `foreach`-Schleife, die über Name, Service, Auflösung und Dauer iteriert. Zu jedem dieser Werte wird nun ein Element angelegt. Da hier ein ganzes Element, also öffnender Tag, Textinhalt und schließender Tag auf einmal erzeugt werden, kann die Methode `writeElement($tag, $content)`²³ verwendet werden. Sie erzeugt ein komplettes Element mit Inhalt. Nach dieser Schleife wird `endElement()`²⁴ aufgerufen. Diese Methode schließt das zuletzt geöffnete Element, in diesem Fall `<show>` mit seinem Gegenstück `</show>`.

Nun ist noch das Element `<shows>` offen, welches durch einen erneuten Aufruf von `endElement()` geschlossen wird. Danach folgt der Aufruf von `endDocument()`²⁵, welches das Dokument als geschlossen markiert. Ein Aufruf von `flush()`²⁶ leert den Speicher und schreibt das Dokument in die Datei.

```

1 $this->writer->startDocument("1.0", "UTF-8");
2 $this->writer->startElement("shows");
3
4 foreach($this->shows as $show) {
5     $this->writer->startElement("show");
6     foreach ($show as $tag => $data) {

```

¹⁹<https://www.php.net/manual/de/xmlwriter.startdocument.php>

²⁰<https://www.php.net/manual/de/xmlwriter.writetd.php>

²¹<https://www.php.net/manual/de/xmlwriter.startelement.php>

²²<https://www.php.net/manual/de/xmlwriter.writeattribute.php>

²³<https://www.php.net/manual/de/xmlwriter.writeelement.php>

²⁴<https://www.php.net/manual/de/xmlwriter.endelement.php>

²⁵<https://www.php.net/manual/de/xmlwriter.enddocument.php>

²⁶<https://www.php.net/manual/de/xmlwriter.flush.php>

```
7     $this->writer->writeElement($tag, $data);
8   }
9   $this->writer->endElement();
10 }
11 $this->writer->endElement();
12
13 $this->writer->endDocument();
14
15 $this->writer->flush();
```

Ausgabe: Der soeben gezeigte Code erzeugt eine XML-Datei mit folgendem Inhalt:

```
<?xml version="1.0" encoding="UTF-8"?>
<shows>
  <show>
    <name>The Mandalorian</name>
    <service>Disney+</service>
    <resolution>2160p</resolution>
    <duration>40</duration>
  </show>
  <show>
    <name>Rick and Morty</name>
    <service>Netflix</service>
    <resolution>1080p</resolution>
    <duration>20</duration>
  </show>
</shows>
```

DOM

Ähnlich wie schon in JavaScript, als das DOM verwendet wurde, um neue HTML-Inhalte zu generieren, kann die *DOM XML-Erweiterung* auch verwendet werden, um neue XML-Dateien zu erstellen oder bestehende zu verändern. Vor allem für Zweites ist DOM unerlässlich, da eine XML-Struktur geladen, an einer Stelle verändert und wieder gespeichert werden kann, ohne die gesamte Struktur neu generieren zu müssen.

DOM ist rein objektorientiert aufgebaut. Das Schreiben verläuft sehr ähnlich zum Parsen (siehe Abschnitt 10.1.2). Der folgende Code stammt aus einer Klasse namens `MyDOMWriter`.

Writer initialisieren und XML erzeugen: Wie auch beim Parsen muss ein Objekt vom Typ `Dom\XMLDocument` angelegt werden, dieses mal jedoch mit dem statischen Konstruktor `createEmpty()`. Es empfiehlt sich, die Versionsnummer (1.0) und die Zeichenkodierung (UTF-8) als Argumente anzugeben. Weiters ist es von Vorteil, die Eigenschaft `formatOutput` auf `true` zu setzen, um Zeilenumbrüche und Einrückungen zu erhalten. Beides wird nicht im Konstruktor, sondern direkt in `generateXML()` erledigt. Da ein DOM-Dokument zunächst im Speicher angelegt und dann befüllt und gespeichert wird, sollte dies zusammen geschehen. Ansonsten würde bei einem mehrmaligen Aufruf von `generateXML()` immer wieder die XML-Struktur in dasselbe DOM-Dokument geschrieben.

Nachdem ein neues (leeres) `XMLDocument`-Objekt angelegt wurde, kann nun begonnen werden, die Baumstruktur darin zu generieren. Zunächst wird das Wurzelement `<shows>` erzeugt. Hierfür kommt `createElement($name)` (analog zu JavaScript) zum Einsatz. Sie erzeugt ein leeres Element. Damit dieses aber auch seinen Weg ins XML-Dokument findet, muss es in der DOM-Struktur platziert werden. Hierfür wird `appendChild($node)` verwendet. Die Methode hängt einen Knoten an einen anderen und gibt den neu angehängten wieder zurück. Das Wurzelement wird also erzeugt und an das neu erzeugte `XMLDocument`-Objekt gehängt. Der Rückgabewert wird in `$shows` gespeichert. An dieses Element werden dann die weiteren angehängt.

Um die einzelnen Shows zu erzeugen, kommt wiederum eine `foreach`-Schleife zum Einsatz. Sie läuft über alle Shows und erzeugt zunächst ein `<show>`-Element und hängt es an das bestehende `<shows>`-Wurzelement. Danach wird über die einzelnen Eigenschaften der Show iteriert und jeweils eigene Elemente mit `createElement()` erzeugt. Als Name dient der Array-Schlüssel, als Wert wird der Wert im Array übergeben und mittels `textContent`-Eigenschaften ins leere Element gesetzt. Danach wird das Element in das `<show>`-Element gehängt.

Schließlich wird die XML-Datei mit der Methode `saveXmlFile($file)` in die angegebene Datei geschrieben.

```

1 use Dom\XMLDocument;
2
3 class MyDOMWriter
4 {
5     private array $shows;
6
7     public function __construct(array $shows)
8     {
9         $this->shows = $shows;
10    }
11
12    public function generateXML(string $file): void
13    {
14        $dom = XMLDocument::createEmpty("1.0", "UTF-8");
15        $dom->formatOutput = true;
16
17        $shows = $dom->appendChild($dom->createElement("shows"));
18
19        foreach ($this->shows as $show) {
20            $showElem = $shows->appendChild($dom->createElement("show"));
21            foreach ($show as $tag => $data) {
22                $showData = $dom->createElement($tag);
23                $showData->textContent = $data;
24                $showElem->appendChild($showData);
25            }
26        }
27
28        $dom->saveXmlFile($file);
29    }
30 }

```

Ausgabe: Die erzeugte Ausgabe gleicht der des XMLWriters:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<shows>
  <show>
    <name>The Mandalorian</name>
    <service>Disney+</service>
    <resolution>2160p</resolution>
    <duration>40</duration>
  </show>
  <show>
    <name>Rick and Morty</name>
    <service>Netflix</service>
    <resolution>1080p</resolution>
    <duration>20</duration>
  </show>
</shows>
```

10.1.4 Diskussion der einzelnen Erweiterungen

Jede der demonstrierten Erweiterungen bietet ihre Vor- und Nachteile. Diese sollen in aller Kürze nochmals zusammen gefasst werden.

Parser

Beim Parsen stehen die Modell, Pull- und objektorientiertes Parsing zur Verfügung. Während sich das erste Modell gut für große Dateien eignen, ist der objektorientierte Vertreter vor allem für den wahlfreien Zugriff zu gebrauchen.

Pull-Parsing stellt einen Mittelweg zwischen veraltetem (und daher ausgelassenem) eventbasiertem und objektorientiertem Parsing dar und kombiniert die Vorteile aus beidem. Einerseits geringer Speicherverbrauch, andererseits Anlehnung an das DOM, was die Verarbeitung der Elemente betrifft.

DOM-Parsing ist vor allem dann interessant, wenn Zugriff auf spezifische Elemente gebraucht wird. Soll ohnehin das gesamte Dokument abgearbeitet werden, stehen der hohe Speicherverbrauch und das Fehlen eines praktischen Iterators dem entgegen.

Writer

Beim Schreiben von XML besteht die Auswahl zwischen dem sequentiellen XMLWriter und einem DOM-basierten Ansatz.

Die XMLWriter-Erweiterung ist modern und objektorientiert und verfolgt einen natürlichen Ansatz. Das Dokument wird von vorne nach hinten aufgebaut, dabei muss die gewollte Struktur jedoch stets im Kopf behalten werden. Ein Stolperstein kann das Vergessen von schließenden Elementen sein, was die Struktur durcheinander bringt.

Die DOM-Erweiterung ist die vom Umfang her mächtigste, aber auch komplizierteste. Sie ist sowohl gut zum Erzeugen von neuem Markup als auch zum Verändern von bestehendem geeignet. Zur Verwendung ist eine gute Vorstellung des DOM-Baums unerlässlich. Da aber Elemente erzeugt und dann hierarchisch aneinander gehängt werden, gibt es weniger Fallstricke im Bezug auf vergessene Elemente und dergleichen.

10.1.5 XML-Komponenten

Auch für die Arbeit mit XML gibt es von der PHP-Community entwickelte Komponenten, die mit Composer einfach installiert werden können.

sabre/xml

Die Bibliothek *sabre/xml*²⁷ folgt so genannten Entwurfsmustern (engl. „design patterns“) und bietet einen strukturierten, objektorientierten Zugriff auf XML – sowohl lesend als auch schreibend. Dabei werden im Hintergrund XMLReader und XMLWriter verwendet, ohne aber deren Methoden verwenden zu müssen. *sabre/xml* ist dabei sehr effizient im Umgang mit XML-Namespaces und versucht generell die Anzahl der Codezeilen, die zu schreiben sind, zu minimieren (vor allem im Vergleich etwa mit DOM).

FluidXML

*FluidXML*²⁸ (installierbar als *servo/fluidxml*) ist eine an das DOM angelehnte XML-Bibliothek. Sie verhält sich nach dem Prinzip des Fluent Programming, das eine natürliche Aneinanderreihung von Methoden vorsieht – quasi als ob Sätze in natürlicher Sprache gesprochen würden. FluidXML greift dabei das Knotenprinzip des DOM auf und ist auch mit ihm vollständig kompatibel, versucht jedoch die manchmal etwas umständlich anmutenden Herangehensweisen des DOM zu vereinfachen.

10.2 JSON lesen und schreiben

Die JavaScript Object Notation – kurz JSON – hat sich neben Formaten wie XML oder YAML schon lange als Datenaustauschformat etabliert. In der Vorlesung Web Fundamentals wurden der Aufbau und die Funktionsweise des Formats erläutert. Auch PHP bietet Funktionen, die in JSON notierte Daten in eine PHP-Datenstruktur umwandeln können bzw. auch den umgekehrten Prozess beherrschen. Diese Funktionen sind streng genommen keine Dateifunktionen, da die JSON-Daten als String eingelesen bzw. generiert werden. Mit Hilfe von `file_get_contents()`²⁹ und `file_put_contents()`³⁰ ist das Lesen und Schreiben von bzw. in Dateien mit einem Zusatzschritt erledigt. Somit bietet sich JSON auch für PHP-Applikationen als Format an, um damit Daten in Dateien zu persistieren.

10.2.1 Lesen von JSON-Dateien

JSON-Daten werden mit der Funktion `json_decode($json [, $assoc])`³¹ eingelesen. Sie verarbeitet den String `$json` und liefert eine PHP-Datenstruktur – im Regelfall ein Objekt, das die JSON-Eigenschaften abbildet. Wird der optionale zweite Parameter `$assoc` auf `true` gesetzt, wird ein assoziatives Array anstelle eines Objekts erzeugt.

²⁷<https://sabre.io/xml/>

²⁸<https://packagist.org/packages/servo/fluidxml>

²⁹<https://www.php.net/manual/de/function.file-get-contents.php>

³⁰<https://www.php.net/manual/de/function.file-put-contents.php>

³¹<https://www.php.net/manual/de/function.json-decode.php>

Das folgende Beispiel verwendet einen Adressbucheintrag in JSON, der zu Demonstrationszwecken etwas mehr Verschachtelung bietet, als das Knödelrezept. Der Eintrag sei in einer Datei `addressbook.json` gespeichert.

```
1 {
2   "firstName": "John",
3   "lastName": "Doe",
4   "age": 25,
5   "address": {
6     "street": "21 Doe St.",
7     "city": "Doeville",
8     "zip": "12345",
9     "state": "NY",
10    "country": "USA"
11  },
12  "phoneNumbers": [
13    {
14      "type": "home",
15      "number": "800 123 456-789"
16    },
17    {
18      "type": "work",
19      "number": "800 999 666-333"
20    }
21  ]
22 }
```

Ausgabe als Objekt

Diese Datei wird nun mit `file_get_contents()` in eine Zeichenkette gelesen. Diese wird der Funktion `json_decode()` übergeben. Das resultierende Objekt wird mittels `var_dump()` ausgegeben.

```
1 $jsonString = file_get_contents("addressbook.json");
2 $jsonData = json_decode($jsonString);
3 var_dump($jsonData);
```

Das Ergebnis ist folgendes Objekt:

```
/var/www/html/wet2vo-examples/vo10/json/jsondecode.php:11:
object(stdClass)[1]
  public 'firstName' => string 'John' (length=4)
  public 'lastName' => string 'Doe' (length=3)
  public 'age' => int 25
  public 'address' =>
    object(stdClass)[2]
      public 'street' => string '21 Doe St.' (length=10)
      public 'city' => string 'Doeville' (length=8)
      public 'zip' => string '12345' (length=5)
      public 'state' => string 'NY' (length=2)
      public 'country' => string 'USA' (length=3)
  public 'phoneNumbers' =>
    array (size=2)
      0 =>
        object(stdClass)[3]
          public 'type' => string 'home' (length=4)
```

```

        public 'number' => string '800 123 456-789' (length=15)
1 =>
    object(stdClass)[4]
        public 'type' => string 'work' (length=4)
        public 'number' => string '800 999 666-333' (length=15)

```

Der Zugriff auf die einzelnen Eigenschaften erfolgt nun mittels der gewohnten PHP-Objekt- bzw. Arraysyntax. Im Folgenden werden Nachname, Straße und die zweite Telefonnummer ausgegeben.

```

1 echo $jsonData->lastName; // Doe
2 echo $jsonData->address->street; // 21 Doe St.
3 echo $jsonData->phoneNumbers[1]->number; // 800 999 666-333

```

Ausgabe als Array

Wird die Funktion mit einem zweiten Parameter, der den Wert `true` hat, aufgerufen, erzeugt `json_decode()` ein assoziatives Array anstelle eines Objekts.

```

1 $jsonString = file_get_contents("addressbook.json");
2 $jsonData = json_decode($jsonString, true);
3 var_dump($jsonData);

```

Dieses sieht wie folgt aus:

```

/var/www/html/wet2vo-examples/vo10/json/jsondecode.php:11:
array (size=5)
    'firstName' => string 'John' (length=4)
    'lastName' => string 'Doe' (length=3)
    'age' => int 25
    'address' =>
        array (size=5)
            'street' => string '21 Doe St.' (length=10)
            'city' => string 'Doeville' (length=8)
            'zip' => string '12345' (length=5)
            'state' => string 'NY' (length=2)
            'country' => string 'USA' (length=3)
    'phoneNumbers' =>
        array (size=2)
            0 =>
                array (size=2)
                    'type' => string 'home' (length=4)
                    'number' => string '800 123 456-789' (length=15)
            1 =>
                array (size=2)
                    'type' => string 'work' (length=4)
                    'number' => string '800 999 666-333' (length=15)

```

Der Zugriff auf die einzelnen Eigenschaften geschieht ausschließlich über die Array-syntax. Es werden wieder Nachname, Straße und die zweite Telefonnummer ausgegeben.

```

1 echo $jsonData["lastName"]; // Doe
2 echo $jsonData["address"]["street"]; // 21 Doe St.
3 echo $jsonData["phoneNumbers"][1]["number"]; // 800 999 666-333/

```

10.2.2 Schreiben von JSON-Dateien

Das Schreiben bzw. Erstellen eines JSON-Datensatzes geschieht mit Hilfe der Funktion `json_encode($value [, $options])`³². Sie akzeptiert mit `$value` jede beliebige PHP-Datenstruktur (ausgenommen Ressourcen). Diese Datenstruktur wird dann in eine valide JSON-Zeichenkette umgewandelt. Mit `$options` können zusätzliche Optionen spezifiziert werden. Hier empfiehlt sich eine Kombination aus `JSON_UNESCAPED_SLASHES` und `JSON_PRETTY_PRINT` um Schrägstriche nicht zu escapen und das Ergebnis schön formatiert aufzubereiten. Letzteres macht beim Schreiben in Dateien umso mehr Sinn, wenn diese von Menschen gelesen werden sollen.

Im folgenden Beispiel werden vier unterschiedliche Datentypen angelegt: ein Objekt, ein indiziertes Array, ein assoziatives Array und eine einfache Ganzzahl. Die Klassendefinition verwendet dabei die in Abschnitt 4.2.4 gezeigte Constructor Property Promotion, die die Definition von Eigenschaften und die Zuweisung von Parametern an selbe deutlich verkürzt.

```
1 class Person
2 {
3     public function __construct(public string $name, public int $age, public string
4         $country) {}
5 }
6 $array = [
7     "Good",
8     "Bad",
9     "Ugly",
10 ];
11
12 $assocArray = [
13     "key" => "value",
14     "otherkey" => "othervalue",
15 ];
16
17 $simpleNumber = 42;
```

Diese werden jeweils mit `json_encode()` in eine JSON-Datenstruktur umgewandelt und danach mit `file_put_contents()` in eine Datei geschrieben.

```
1 $jsonObject = json_encode(
2     new Person("John Doe", 34, "Doeland"),
3     JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT
4 );
5 $jsonArray = json_encode(
6     $array,
7     JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT
8 );
9 $jsonAssocArray = json_encode(
10    $assocArray,
11    JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT
12 );
13 $jsonInteger = json_encode(
14    $simpleNumber,
15    JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT
```

³²<https://www.php.net/manual/de/function.json-encode.php>

```
16 );  
17  
18 file_put_contents("object.json", $jsonObject);  
19 file_put_contents("array.json", $jsonArray);  
20 file_put_contents("assocarray.json", $jsonAssocArray);  
21 file_put_contents("integer.json", $jsonInteger);
```

Das Ergebnis sieht wie folgt aus. PHP-Objekte und assoziative Arrays erzeugen JSON-Objekte. Ein indiziertes Array erzeugt ein JSON-Array. Eine einfache Integerzahl wird auch als solche in JSON dargestellt. Um also die gewünschte JSON-Struktur zu erhalten (in der Regel ein Objekt), muss auch die entsprechende PHP-Datenstruktur gewählt werden.

```
** object.json **  
{  
  "name": "John Doe",  
  "age": 34,  
  "country": "Doeland"  
}  
  
** array.json **  
[  
  "Good",  
  "Bad",  
  "Ugly"  
]  
  
** assocarray.json **  
{  
  "key": "value",  
  "otherkey": "othervalue"  
}  
  
** integer.json **  
42
```

10.2.3 JSON Fehlerbehandlung

Sind beim Kodieren oder Dekodieren von JSON-Daten Fehler aufgetreten, so können diese mit Hilfe der Funktionen `json_last_error()`³³ oder `json_last_error_msg()`³⁴ identifiziert werden. Erstere Funktion gibt dabei den Fehlercode als Integerzahl zurück, die mit Hilfe von Konstanten interpretiert werden muss. Einfacher ist daher die Verwendung der zweiten Funktion, die diese Interpretation bereits selbst vornimmt und eine Meldung als String zurück gibt. Ist kein Fehler passiert, wird „No error“ ausgegeben.

10.2.4 JSON validieren

Seit PHP 8.3 steht mit `json_validate()`³⁵ eine Funktion zur Verfügung, die eine JSON-Datenstruktur auf Korrektheit überprüft. Sie kann eingesetzt werden, wenn es nicht

³³ <https://www.php.net/manual/de/function.json-last-error.php>

³⁴ <https://www.php.net/manual/de/function.json-last-error-msg.php>

³⁵ <https://www.php.net/manual/de/function.json-validate.php>

um den Inhalt geht, sondern nur darum, ob die JSON-Syntax eingehalten wurde. Da auch `json_decode()` eine Validierung durchführt, sollten die beiden Funktionen nicht hintereinander verwendet werden, weil dies ein (unnötiges) doppeltes Verarbeiten der JSON-Daten zur Folge hat.

```
1 $jsonString = file_get_contents("addressbook.json");
2 $jsonIsValid = json_validate($jsonString);
3
4 echo $jsonIsValid ? "JSON structure is valid." : "JSON structure is not valid.";
```

10.3 Grafiken mit der GD Graphics Library

Webseiten bestehen in der Regel nicht nur aus Text, meist sind auch *Bilder* in irgendeiner Form involviert. Logos, Buttons, Werbebanner oder Icons sind nur einige Beispiele dafür. Viele, wenn nicht der Großteil dieser Bilder sind rein statisch und wurden mit Werkzeugen wie Photoshop erstellt. Andere jedoch müssen dynamisch generiert werden, da ihr Inhalt von bestimmten Faktoren abhängt. Beispiele hierfür sind Grafiken zum aktuellen Aktienkurs oder personalisierte Werbebanner.

PHP ermöglicht die dynamische Erzeugung von Grafiken durch diverse *Grafikbibliotheken*. Dies sind ganz allgemein Softwarepakete, die es ermöglichen, Grafikausgabe am Bildschirm darzustellen oder in einer Datei zu speichern, d. h. Grafiken zu erstellen.

Bekannte Vertreter sind *ImageMagick*³⁶, *GraphicsMagick*³⁷ oder auch die GD Graphics Library, welche hier für einige wichtige Funktionalitäten vorgestellt wird.

10.3.1 Allgemeines zur GDlib

Die *GD Graphics Library*³⁸, auch *GD Library*, *GDlib*, *LibGD* oder kurz *GD*, ist eine Grafikbibliothek geschrieben in C, entwickelt von Thomas Boutell und mittlerweile fortgeführt von Pierre Joye. Der Name „GD“ stand ursprünglich für „GIF Draw“, nachdem aber aufgrund patentrechtlicher Umstände die GIF-Unterstützung kurzzeitig entfernt werden musste, wurde der Name auf „Graphics Draw“ geändert.

Die Bibliothek kann mit den bekannten Formaten JPEG, GIF, PNG, WebP, AVIF, TGA und BMP umgehen (teilweise müssen dazu aber die entsprechenden Bibliotheken im Betriebssystem installiert sein). Daneben werden noch WBMP, XBM und XPM sowie die eigenen Formate GD und GD2 unterstützt. Dies ist im Vergleich zu ImageMagick eine eher bescheidene Auswahl an verarbeitbaren Formaten, der große Vorteil von GD liegt jedoch in der Tatsache, dass die Bibliothek seit PHP 4.3.0 ein Standardbestandteil der Distribution ist und somit auf allen aktuellen PHP-Systemen verfügbar ist.

Zusätzlich zur guten Verfügbarkeit zeichnet sich die GD Graphics Library durch ihre einfache Verwendung aus. Einen Überblick über alle GD-Funktionalitäten bietet das PHP-Handbuch³⁹.

³⁶<https://imagemagick.org/>

³⁷<http://www.graphicsmagick.org/>

³⁸<https://libgd.github.io/>

³⁹<https://www.php.net/manual/de/book.image.php>

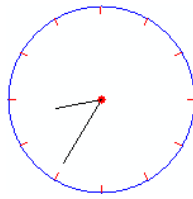


Abbildung 10.3: Die Grafik der Uhr wird mittels GD Library erzeugt und zeigt somit immer die richtige Uhrzeit an. Quelle: [2].

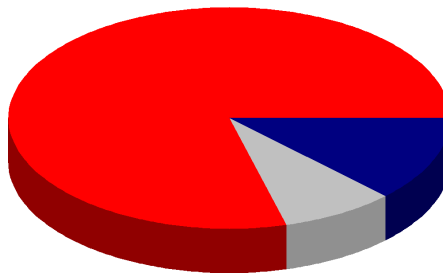


Abbildung 10.4: Das dynamische Kuchendiagramm ist ideal, um Daten anzuzeigen, die sich verändern können. Quelle: [1].

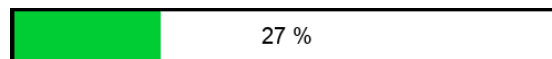


Abbildung 10.5: Die Fortschrittsanzeige wird dynamisch durch GD berechnet und erzeugt.

Einsatzgebiete

Die Einsatzgebiete für dynamische Grafiken im Web sind vielfältig. Zunächst ist die *Erzeugung* von Grafiken ein wichtiger Punkt. Wann immer nicht bekannt ist, wie eine Grafik zu einem gegebenen Zeitpunkt aussehen wird oder wenn die Grafik auf sich verändernden Daten beruht, muss die Grafik dynamisch erzeugt werden. Ein Beispiel ist etwa eine Uhr, wie in Abbildung 10.3 ersichtlich. Soll eine analoge Uhr, die immer die richtige Uhrzeit anzeigt, grafisch dargestellt werden, würden 720 (mit Sekunden sogar 43.200) verschiedene Bilder benötigt. Mit einer dynamisch erzeugten Grafik wird die Position der einzelnen Zeiger einfach anhand der aktuellen Zeit gesetzt.

Ein weiteres Beispiel ist die Erzeugung von Diagrammen, etwa Balken- oder Kuchendiagrammen. Die Verteilung des Diagramms kann hier etwa auf einem sich verändernden Datensatz beruhen. Dies macht es schwer möglich, das Diagramm im Vorhinein zu erstellen. Abbildung 10.4 zeigt ein 3D-Kuchendiagramm, das mit GD erzeugt wurde.

Ein weiterer Anwendungsfall sind Fortschrittsanzeigen. Wann immer es nötig ist, einen Fortschritt oder Verlauf anzuzeigen, können mit GD grafische Fortschrittsanzeigen erzeugt werden. Abbildung 10.5 stellt ein Beispiel dar.

Schließlich sind Grafikbibliotheken wie GD auch ideal, um bestehende Bilder zu bearbeiten. Häufige Szenarien sind hierbei das Verkleinern von Bildern, das Entnehmen

eines Ausschnitts oder auch das Konvertieren von einem Grafikformat in ein anderes. Dies wird auch der hier demonstrierte Einsatzzweck sein.

Die folgenden Abschnitte erläutern nun, wie mittels der GD Graphics Library Bilder erstellt und verarbeitet werden.

10.3.2 Neue Bilder erstellen

Jedem neuen Bild liegt eine *Zeichenfläche* (engl. „canvas“) zugrunde. Diese ist pixelbasiert und hat eine feste Größe. Die Zeichenfläche selbst ist noch nicht sichtbar und enthält noch nichts (genauer gesagt ist sie komplett schwarz). Sie ist vielmehr eine einfache Repräsentation des Bildes, auf der dann gearbeitet wird. Die Zeichenfläche wird als Objekt vom Typ `GdImage` erstellt. Damit das Bild dann wirklich angezeigt wird, muss eine eigene Funktion aufgerufen werden (siehe Abschnitt 10.3.4).

Um nun eine neue Zeichenfläche zu erzeugen, bietet GD zwei Funktionen:

- `imagecreate($width, $height)` und
- `imagecreatetruecolor($width, $height)`.

`imagecreate`

Die Funktion `imagecreate($width, $height)`⁴⁰ erzeugt eine neue Zeichenfläche mit 256 Farben. Die Funktion gibt das `GdImage`-Objekt zurück, das in weiterer Folge für alle Operationen benötigt wird.

Der folgende Code erzeugt eine Zeichenfläche mit 200×200 Pixeln und 256 möglichen Farben. Das Objekt ist dann über `$image` verfügbar.

```
1 $image = imagecreate(200, 200);
```

`imagecreatetruecolor`

`imagecreatetruecolor($width, $height)`⁴¹ erzeugt ebenso eine Zeichenfläche, bietet jedoch ein Truecolor-Bild mit 16,7 Millionen Farben. Die Syntax verlangt ebenfalls die Angabe einer Breite und Höhe, zurückgegeben wird wieder ein `GdImage`-Objekt.

Im Folgenden wird eine Zeichenfläche mit 16,7 Millionen Farben und ebenfalls 200×200 Pixeln erzeugt:

```
1 $image = imagecreatetruecolor(200, 200);
```

Generell sollte immer `imagecreatetruecolor()` verwendet werden, um die komplette Farbpalette zur Verfügung zu haben. Eine Ausnahme ist die Erzeugung von GIF-Bildern. Diese unterstützen ohnehin nur 256 Farben, somit ist auch das Anlegen eines Truecolor-Canvas nicht zielführend.

10.3.3 Bestehende Bilder öffnen

Soll eine bestehende Bilddatei *geöffnet* werden, kann das Anlegen einer leeren Zeichenfläche entfallen. Vielmehr gibt es eigene Funktionen, die ein `GdImage`-Objekt mit den

⁴⁰<https://www.php.net/manual/de/function.imagecreate.php>

⁴¹<https://www.php.net/manual/de/function.imagecreatetruecolor.php>

Daten einer bestehenden Bilddatei zurückgegeben können. Je nach Dateityp kommt eine andere Funktion zum Einsatz:

- `imagecreatefrompng($filename)`⁴²,
- `imagecreatefromjpeg($filename)`⁴³,
- `imagecreatefromgif($filename)`⁴⁴,
- `imagecreatefromwebp($filename)`⁴⁵,
- `imagecreatefromavif($filename)`⁴⁶,
- `imagecreatefromtga($filename)`⁴⁷,
- `imagecreatefrombmp($filename)`⁴⁸,
- `imagecreatefromwbmp($filename)`⁴⁹,
- `imagecreatefromxbm($filename)`⁵⁰,
- `imagecreatefromxpm($filename)`⁵¹,
- `imagecreatefromgd($filename)`⁵²,
- `imagecreatefromgd2($filename)`⁵³.

Im folgenden Code wird die bestehende Datei `bild1.png` geöffnet. Die Daten (d. h. die Zeichenfläche) stehen dann wieder als Objekt in der Variable `$image` zur Verfügung. Der Öffnungsprozess ist derselbe für alle anderen Formate. Die Formate GD und GD2 sind eigene GD Library Formate, die verwendet werden können, wenn eine große Menge an Bildern zwischengespeichert und wieder geöffnet werden muss, da sie native Formate der Bibliothek sind.

```
1 $image = imagecreatefrompng("bild1.png");
```

Eine weitere Möglichkeit, verschiedenste bestehende Bilder zu öffnen, stellt die Funktion `imagecreatefromstring($image)`⁵⁴ dar. Als Parameter erwartet sie einen so genannten Datenstrom mit Bildinhalten, also einen String, der die Dateiinhalte repräsentiert. Ein solcher kann, wie bereits bei JSON in Abschnitt 10.2 bereits erwähnt, mit der Funktion `file_get_contents($filename)` erhalten werden. Werden beide Funktionen kombiniert, kann diese zum Öffnen verschiedenster Dateitypen (sofern unterstützt) verwendet werden:

```
1 $image = imagecreatefromstring(file_get_contents("bild1.jpg"));
```

⁴²<https://www.php.net/manual/de/function.imagecreatefrompng.php>

⁴³<https://www.php.net/manual/de/function.imagecreatefromjpeg.php>

⁴⁴<https://www.php.net/manual/de/function.imagecreatefromgif.php>

⁴⁵<https://www.php.net/manual/de/function.imagecreatefromwebp.php>

⁴⁶<https://www.php.net/manual/de/function.imagecreatefromavif.php>

⁴⁷<https://www.php.net/manual/de/function.imagecreatefromtga.php>

⁴⁸<https://www.php.net/manual/de/function.imagecreatefrombmp.php>

⁴⁹<https://www.php.net/manual/de/function.imagecreatefromwbmp.php>

⁵⁰<https://www.php.net/manual/de/function.imagecreatefromxbm.php>

⁵¹<https://www.php.net/manual/de/function.imagecreatefromxpm.php>

⁵²<https://www.php.net/manual/de/function.imagecreatefromgd.php>

⁵³<https://www.php.net/manual/de/function.imagecreatefromgd2.php>

⁵⁴<https://www.php.net/manual/de/function.imagecreatefromstring.php>

10.3.4 Bilder ausgeben

Um Bilder *auszugeben*, gibt es zwei Möglichkeiten. Entweder wird das Bild dem Browser direkt zur Anzeige übergeben oder in einer Datei gespeichert.

Gleichgültig, auf welche Art die Ausgabe dann schlussendlich erfolgt, muss zuvor die Entscheidung getroffen werden, in welchem Grafikformat die Ausgabe erfolgen soll. Ähnlich wie beim Einlesen von bestehenden Bildern gibt es dann auch für die Ausgabe eine Funktion pro Grafikformat (TGA- und XPM-Ausgabe werden jedoch nicht unterstützt):

- `imagepng($image [, $filename])`⁵⁵,
- `imagejpeg($image [, $filename])`⁵⁶,
- `imagegif($image [, $filename])`⁵⁷,
- `imagewebp($image [, $filename])`⁵⁸,
- `imageavif($image [, $filename])`⁵⁹,
- `imagebmp($image [, $filename])`⁶⁰,
- `imagewbmp($image [, $filename])`⁶¹,
- `imagexbm($image [, $filename])`⁶²,
- `imagegd($image [, $filename])`⁶³,
- `imagegd2($image [, $filename])`⁶⁴.

Im Browser

Wurde ein Ausgabeformat gewählt, in dem das Bild im Browser angezeigt werden soll, so muss die jeweilige Funktion mit einem Parameter – nämlich dem `GdImage`-Objekt – aufgerufen werden. *Davor* jedoch muss dem Browser mitgeteilt werden, dass es sich bei der angezeigten Datei um eine Bilddatei im jeweils gewählten Format handelt.

Wird das Bild direkt im Browser ausgegeben, d. h. wird die PHP-Datei aufgerufen, deren alleiniger Zweck es ist, ein Bild zu erzeugen und darzustellen, so muss mit Hilfe der Funktion `header()` der **Content-Type-Header** auf diesen Bildtyp gesetzt werden. Somit weiß der Browser, wenn er die Response vom Server erhält, dass ein Bild und keine HTML-Seite retour kommt.

Der folgende Code wird in der Datei `emptyimage.php` gespeichert. Es wird ein neues Bild angelegt und danach sofort ausgegeben. Da keinerlei Zeichenoperationen geschehen, wird lediglich ein schwarzes Rechteck angezeigt, da ein neues Bild nur schwarz enthält. Das Bild wird als PNG ausgegeben (durch die Verwendung von `imagepng()`), zuvor

⁵⁵<https://www.php.net/manual/de/function.imagepng.php>

⁵⁶<https://www.php.net/manual/de/function.imagejpeg.php>

⁵⁷<https://www.php.net/manual/de/function.imagegif.php>

⁵⁸<https://www.php.net/manual/de/function.imagewebp.php>

⁵⁹<https://www.php.net/manual/de/function.imageavif.php>

⁶⁰<https://www.php.net/manual/de/function.imagebmp.php>

⁶¹<https://www.php.net/manual/de/function.imagewbmp.php>

⁶²<https://www.php.net/manual/de/function.imagexbm.php>

⁶³<https://www.php.net/manual/de/function.imagegd.php>

⁶⁴<https://www.php.net/manual/de/function.imagegd2.php>

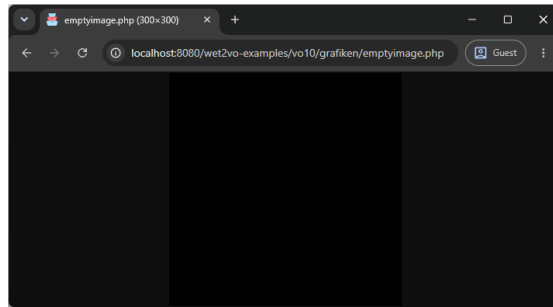


Abbildung 10.6: Ein leeres Bild wird ausgegeben. Die Zeichenfläche ist standardmäßig schwarz. Die Ausgabe erfolgt als 300×300 Pixel PNG-Bild. Interessant ist die URL-Zeile des Browsers: Hier wird die PHP-Datei (und keine Grafikdatei) aufgerufen.

wird mit der Funktion `header()` der Content-Type auf `image/png` gesetzt. Abbildung 10.6 zeigt die Ausgabe.

```
1 $image = imagecreatetruecolor(300, 300);  
2  
3 header("Content-type: image/png");  
4 imagepng($image);
```

Als Datei

Das Schreiben einer Bilddatei gestaltet sich ganz ähnlich. Hier kommt jedoch bei den Ausgabemethoden ein zweiter Parameter hinzu, der den zu verwendenden Dateinamen angibt. Das Setzen des Content-Type Headers kann entfallen, da die PHP-Datei in der Response ja kein Bild zurück liefert, sondern dieses direkt auf die Festplatte schreibt.

Die Datei `writeimage.php` erzeugt ebenfalls ein 300×300 Pixel großes Bild, ohne (d.h. nur mit schwarzem) Inhalt. Die Ausgabe wird jedoch direkt in die Datei `emptyimage.png` geschrieben. Wird also `writeimage.php` im Browser aufgerufen, so wird nichts angezeigt, stattdessen wird aber eine PNG-Datei im aktuellen Verzeichnis erstellt.

```
1 $image = imagecreatetruecolor(300, 300);  
2  
3 imagepng($image, "emptyimage.png");
```

10.3.5 Zeichenfunktionen

Um mit Hilfe der GD-Library dynamisch Inhalte zu erstellen, können die diversen Zeichenfunktionen verwendet werden. Diese seien an dieser Stelle jedoch nur kurz aufgelistet:

- Farben definieren: `imagecolorallocate($image, $red, $green, $blue)`⁶⁵,
- Linienstärke: `imagesetthickness($image, $thickness)`⁶⁶,

⁶⁵ <https://www.php.net/manual/de/function.imagecolorallocate.php>

⁶⁶ <https://www.php.net/manual/de/function.imagesetthickness.php>

- Linienstil: `imagesetstyle($image, $style)`⁶⁷,
- Einzelne Pixel: `imagesetpixel($image, $x, $y, $color)`⁶⁸,
- Linien: `imageline($image, $x1, $y1, $x2, $y2, $color)`⁶⁹,
- Rechtecke: `imagerectangle($image, $x1, $y1, $x2, $y2)`⁷⁰,
- Gefüllte Rechtecke: `imagefilledrectangle($image, $x1, $y1, $x2, $y2)`⁷¹,
- Polygone: `imagepolygon($image, $points, $color)`⁷²,
- Gefüllte Polygone: `imagefilledpolygon($image, $points, $color)`⁷³,
- Kreisbögen: `imagearc($image, $cx, $cy, $w, $h, $s, $e, $c)`⁷⁴,
- Gefüllte Kreisbögen: `imagefilledarc($image, $cx, $cy, $w, $h, $s, $e, $c, $st)`⁷⁵,
- Ellipsen: `imageellipse($image, $cx, $cy, $w, $h, $c)`⁷⁶,
- Gefüllte Ellipsen: `imagefilledellipse($image, $cx, $cy, $w, $h, $c)`⁷⁷,
- Text mit TrueType-Schriften: `imagefttext($image, $size, $angle, $x, $y, $color, $font, $text)`⁷⁸

10.3.6 Bilder kopieren

Die GD Graphics Library ermöglicht es, Inhalte eines Bildes in ein zweites zu *kopieren*. Es kann das gesamte Bild oder nur ein Ausschnitt kopiert werden. Die Funktion `imagecopy($dstIm, $srcIm, $dstX, $dstY, $srcX, $srcY, $srcW, $srcH)`⁷⁹ hat die folgenden Parameter:

- `$dstIm`: Das Zielbild, in das kopiert werden soll.
- `$srcIm`: Das Quellbild, aus dem die Inhalte entnommen werden sollen.
- `$dstX`: Die *x*-Koordinate im Zielbild.
- `$dstY`: Die *y*-Koordinate im Zielbild.
- `$srcX`: Die *x*-Koordinate im Quellbild.
- `$srcY`: Die *y*-Koordinate im Quellbild.
- `$srcW`: Die Breite des Ausschnitts im Quellbild.
- `$srcH`: Die Höhe des Ausschnitts im Quellbild.

Abbildung 10.7 illustriert, wie ein Ausschnitt aus dem Quellbild entnommen und im Zielbild eingefügt wird.

⁶⁷<https://www.php.net/manual/de/function.imagesetstyle.php>

⁶⁸<https://www.php.net/manual/de/function.imagesetpixel.php>

⁶⁹<https://www.php.net/manual/de/function.imageline.php>

⁷⁰<https://www.php.net/manual/de/function.imagerectangle.php>

⁷¹<https://www.php.net/manual/de/function.imagefilledrectangle.php>

⁷²<https://www.php.net/manual/de/function.imagepolygon.php>

⁷³<https://www.php.net/manual/de/function.imagefilledpolygon.php>

⁷⁴<https://www.php.net/manual/de/function.imagearc.php>

⁷⁵<https://www.php.net/manual/de/function.imagefilledarc.php>

⁷⁶<https://www.php.net/manual/de/function.imageellipse.php>

⁷⁷<https://www.php.net/manual/de/function.imagefilledellipse.php>

⁷⁸<https://www.php.net/manual/de/function.imagefttext.php>

⁷⁹<https://www.php.net/manual/de/function.imagecopy.php>

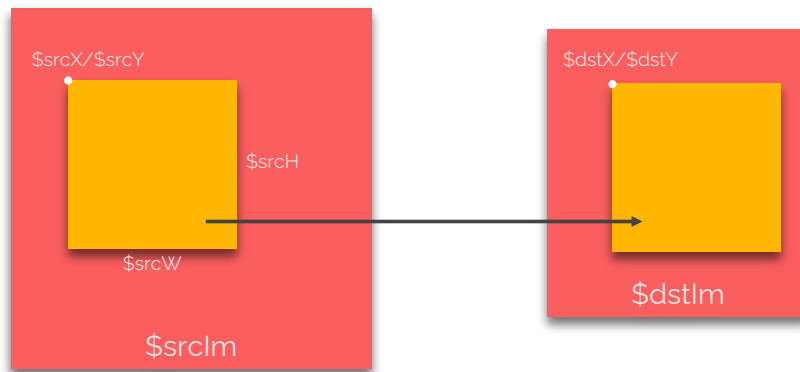


Abbildung 10.7: Ein Ausschnitt aus dem Quellbild wird ins Zielbild kopiert.

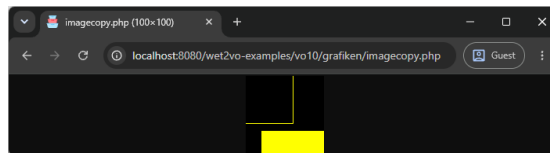


Abbildung 10.8: Ein Ausschnitt aus dem 300×300 Pixel großen Bild wird in ein neues, 100×100 Pixel großes Bild kopiert.

Der folgende Code erstellt zunächst mit zwei der Zeichenfunktionen zwei Rechtecke. Es wird dann, beginnend bei den Koordinaten (50, 50) ein 100×100 Pixel großer Ausschnitt entnommen und in ein neues, ebenfalls 100×100 großes Bild kopiert. Abbildung 10.8 zeigt das Ergebnis.

```

1 $image1 = imagecreatetruecolor(300, 300);
2 $image2 = imagecreatetruecolor(100, 100);
3
4 $yellow = imagecolorallocate($image1, 255, 255, 0);
5
6 imagerectangle($image1, 10, 10, 110, 110, $yellow);
7 imagefilledrectangle($image1, 70, 120, 280, 250, $yellow);
8 imagecopy($image2, $image1, 0, 0, 50, 50, 100, 100);
9
10 header("Content-type: image/png");
11 imagepng($image2);

```

10.3.7 Bildgröße ändern

Analog zum Kopieren funktioniert auch die Änderung der *Bildgröße*. Eigentlich handelt es sich auch hierbei um einen Kopiervorgang, jedoch ist beim Einfügen eine Größenänderung des Zielbereichs möglich, sodass der Ausschnitt entweder vergrößert oder verkleinert dargestellt werden kann.

Zwei Funktionen stehen für diesen Vorgang zur Verfügung:

- `imagecopyresampled($dstIm, $srcIm, $dstX, $dstY, $srcX, $srcY,`

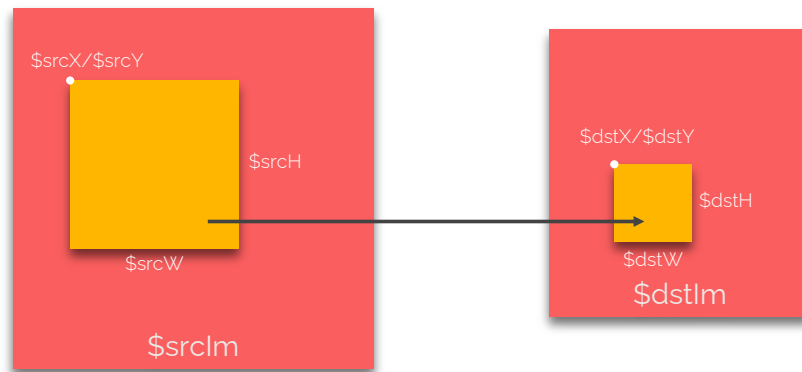


Abbildung 10.9: Ein Ausschnitt aus dem Quellbild wird ins Zielbild kopiert und dabei skaliert.

`$dstW, $dstH, $srcW, $srcH`)⁸⁰ und

- `imagecopyresized($dstIm, $srcIm, $dstX, $dstY, $srcX, $srcY, $dstW, $dstH, $srcW, $srcH)`.⁸¹

Beide weisen eine ähnliche Funktionsweise auf, `imagecopyresampled()` ist jedoch vorzuziehen, da beim Skalieren die einzelnen Farbwerte interpoliert werden, was zu einem visuell besseren Ergebnis führt. Die Parameter beider Funktionen lauten wie folgt:

- `$dstIm`: Das Zielbild, in das kopiert werden soll.
- `$srcIm`: Das Quellbild, aus dem die Inhalte entnommen werden sollen.
- `$dstX`: Die x -Koordinate im Zielbild.
- `$dstY`: Die y -Koordinate im Zielbild.
- `$srcX`: Die x -Koordinate im Quellbild.
- `$srcY`: Die y -Koordinate im Quellbild.
- `$dstW`: Die Breite des Ausschnitts im Zielbild.
- `$dstH`: Die Höhe des Ausschnitts im Zielbild.
- `$srcW`: Die Breite des Ausschnitts im Quellbild.
- `$srcH`: Die Höhe des Ausschnitts im Quellbild.

Abbildung 10.9 zeigt, wie ein Ausschnitt aus dem Quellbild entnommen und skaliert im Zielbild eingefügt wird.

Der folgende Code erstellt wiederum zwei Rechtecke. Es wird nun aber das gesamte, 300×300 Pixel große Bild entnommen und in ein neues, 100×100 großes Bild skaliert eingefügt. Abbildung 10.10 zeigt das Ergebnis.

```
1 $image1 = imagecreatetruecolor(300, 300);
2 $image2 = imagecreatetruecolor(100, 100);
3
4 $yellow = imagecolorallocate($image1, 255, 255, 0);
```

⁸⁰<https://www.php.net/manual/de/function.imagecopyresampled.php>

⁸¹<https://www.php.net/manual/de/function.imagecopyresized.php>



Abbildung 10.10: Das gesamte, 300×300 Pixel großes Quellbild wird in das 100×100 Pixel große Zielbild kopiert und dabei skaliert.

```

5
6 imagerectangle($image1, 10, 10, 110, 110, $yellow);
7 imagefilledrectangle($image1, 70, 120, 280, 250, $yellow);
8
9 imagecopyresampled($image2, $image1, 0, 0, 0, 0, 100, 100, 300, 300);
10
11 header("Content-type: image/png");
12 imagepng($image2);

```

10.3.8 Bilder rotieren

Um Bilder zu *rotieren*, steht `imagerotate($image, $angle, $bgcolor)`⁸² zur Verfügung. Dabei wird das Bild um seinen Mittelpunkt um den in `$angle` spezifizierten Winkel rotiert. Mit `$bgcolor` wird die Farbe der Region bestimmt, die nach der Rotation sichtbar wird.

Die Funktion gibt eine neue `GdImage`-Instanz zurück. Die Größe des neuen Bildes kann sich verändern, denn das Bild wird notfalls erweitert und nicht abgeschnitten.

Der folgende Code rotiert zwei erstellte Rechtecke um 45 Grad. Das resultierende Bild ist nicht mehr 300×300 Pixel sondern 426×426 Pixel groß und in Abschnitt 10.11 ersichtlich.

```

1 $width = $height = 300;
2 $image = imagecreatetruecolor($width, $height);
3
4 $yellow = imagecolorallocate($image, 255, 255, 0);
5 $black = imagecolorallocate($image, 0, 0, 0);
6
7 imagerectangle($image, 10, 10, 110, 110, $yellow);
8 imagefilledrectangle($image, 70, 120, 280, 250, $yellow);
9
10 $image = imagerotate($image, 45, $black);
11
12 header("Content-type: image/png");
13 imagepng($image);

```

10.4 PDFs erzeugen

PDF, das *Portable Document Format* von Adobe, ist der globale Standard für plattformunabhängige Dokumente. Es wurde im Jahr 1992 erstmals veröffentlicht und wird bis

⁸²<https://www.php.net/manual/de/function.imagerotate.php>

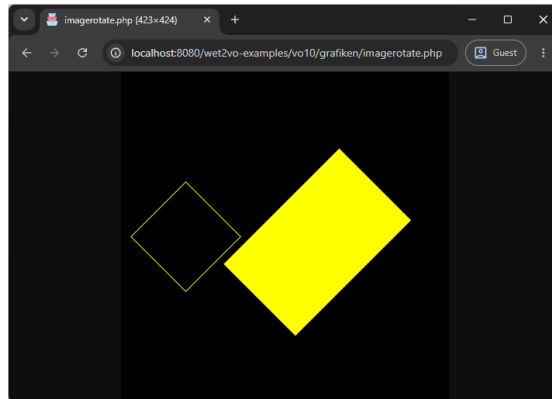


Abbildung 10.11: Das ursprüngliche Bild wurde um 45° rotiert und dadurch von 300×300 Pixel auf 423×424 Pixel vergrößert.

heute aktiv weiterentwickelt. Die aktuelle Version ist 2.0, welche 2017 zusammen mit der Software Adobe Acrobat DC veröffentlicht wurde.

PDF-Dokumente können Text, Grafiken (Vektor- und Rastergrafiken) oder auch andere, multimediale Inhalte wie Videos enthalten. Das PDF-Format bedient sich dabei einer Untermenge der vektorbasierten Seitenbeschreibungssprache PostScript, mit der die Ausgabe von Inhalten layoutgetreu möglich ist. Das Einbetten dieser Inhalte, sowie auch damit verbundener Dinge wie Schriften in eine einzige Datei und die Unabhängigkeit von Ausgabegeräten wie Druckern hat die enorme Verbreitung von PDF zunächst überhaupt ermöglicht und dann in weiterer Folge stark gefördert.

In der modernen Webentwicklung werden PDFs (wie Rechnungen, Tickets oder Reports) kaum noch mühsam über das Platzieren von Elementen an x/y -Koordinaten erstellt. Stattdessen hat sich ein effizienterer Workflow etabliert: Das Dokument wird als reguläres HTML/CSS-Template entworfen und anschließend serverseitig in ein PDF konvertiert.

Der PHP-Core bietet hierfür keine Funktionalitäten. Auch offizielle Erweiterungen existieren nicht, weshalb auf Komponenten zurückgegriffen werden muss. Die derzeit populärste und am aktivsten gepflegte PHP-Bibliothek für diesen Zweck ist **Dompdf**⁸³. Dompdf ist ein (größtenteils) CSS 2.1-konformer HTML-Renderer in PHP, der HTML-Strings einliest und daraus strukturierte PDF-Dateien generiert.

10.4.1 Dompdf initialisieren und Grundlagen

Die Bibliothek wird zunächst wie gewohnt über Composer installiert:

```
1 composer require dompdf/dompdf
```

Die grundlegende Verwendung von Dompdf ist in drei logische Schritte unterteilt: Laden des HTML-Codes, Konfiguration des Seitenformats und Rendern/Ausgeben des PDFs.

Nach dem Einbinden des Autoloaders wird ein Objekt der Klasse Dompdf erzeugt.

⁸³<https://github.com/dompdf/dompdf>

Die Methode `loadHtml()`⁸⁴ nimmt den zu rendernden HTML-String entgegen. Mit `setPaper()`⁸⁵ wird das Format (meist „A4“) und die Ausrichtung („portrait“ für Hochformat oder „landscape“ für Querformat) festgelegt. Der Aufruf von `render()`⁸⁶ startet schließlich den Konvertierungsprozess.

```
1 use Dompdf\Dompdf;
2
3 require __DIR__ . "/vendor/autoload.php";
4
5 $dompdf = new Dompdf();
6
7 $dompdf->loadHtml("<h1>Hello PDF World!</h1><p>My first PDF with Dompdf!</p>");
8
9 $dompdf->setPaper("A4", "portrait");
10
11 $dompdf->render();
```

Ausgabe des PDFs

Nachdem das PDF gerendert wurde, muss es ausgeliefert werden. Dompdf bietet hierfür die Methode `stream($filename, $options)`⁸⁷. Der Parameter `$options` ist ein Array, in dem das Verhalten des Browsers gesteuert werden kann:

- `["Attachment" => false]`: Das PDF wird direkt im Browser (im integrierten PDF-Viewer) *angezeigt* (Inline-Anzeige).
- `["Attachment" => true]`: Der Browser erzwingt einen *Download* der Datei.

Soll das PDF nicht an den Browser gesendet, sondern direkt als Datei auf dem Webserver gespeichert werden, kann der erzeugte Binärcode des PDFs mit der Methode `output()` ausgelesen und über die native PHP-Funktion `file_put_contents()` auf die Festplatte geschrieben werden:

```
1 // Direkt im Browser anzeigen
2 $dompdf->stream("hallo-welt.pdf", ["Attachment" => false]);
3
4 // Am Server speichern (überschreibt bestehende Dateien)
5 file_put_contents(__DIR__ . "/hallo-welt.pdf", $dompdf->output());
```

Anpassen der Optionen

Dompdf erlaubt es, gewisse Standardeinstellungen über ein eigenes Konfigurationsobjekt vom Typ `Dompdf\Options` zu definieren und anzuwenden. Dieses Objekt wird zuerst angelegt und dann bei der Erstellung des `Dompdf`-Objekts als Parameter mitgegeben. Folgende Optionen können u. a. als Methoden gesetzt werden:

- `setDefaultFont()`: Erlaubt die Angabe einer Standardschriftart. Dompdf kennt die Optionen „serif“, „sans-serif“ oder „monospace“. Weitere Schriftarten müssen als TTF-Dateien im Projekt inkludiert und über `setFontDir()` verfügbar gemacht werden.

⁸⁴<https://github.com/dompdf/dompdf/wiki/Usage#loadhtml>

⁸⁵<https://github.com/dompdf/dompdf/wiki/Usage#setpaper>

⁸⁶<https://github.com/dompdf/dompdf/wiki/Usage#render>

⁸⁷<https://github.com/dompdf/dompdf/wiki/Usage#stream>

- `setIsPdfAEnabled()`: Erlaubt die Erzeugung von PDF/A-Dokumenten (schreibgeschützte Archivdokumente).
- `setPdfBackend()`: Ermöglicht die Angabe des Backends, das die Erzeugung des PDFs übernimmt. Der Standard ist „Cpdf“.

Um im Hello World-Beispiel die Standardschriftfamilie auf „Helvetica“ zu setzen, ist folgender Code nötig:

```
1 use Dompdf\Dompdf;
2 use Dompdf\Options;
3
4 $options = new Options();
5 $options->setDefaultFont("Helvetica");
6
7 $dompdf = new Dompdf($options);
8
9 // PDF-Erstellung...
```

Metadaten setzen und Zugriff auf den Canvas

PDF-Dokumente enthalten Metadaten. Dies sind zum einen der Titel, zum anderen aber auch Dinge wie der*die Autor*in, das Thema des PDFs, Schlüsselwörter oder auch die Software, welche das PDF erzeugt hat. Diese Dinge können im `Dompdf`-Objekt über die Methode `addInfo()` gesetzt werden. Im Beispiel sind dies Titel, Autor*in, Betreff, Schlüsselwörter und die erstellende Applikation:

```
1 // PDF rendern
2 $dompdf->render();
3
4 // Metadaten setzen
5 $dompdf->addInfo("Title", "Dompdf Example");
6 $dompdf->addInfo("Author", "Wolfgang Hochleitner");
7 $dompdf->addInfo("Subject", "Dompdf Demo");
8 $dompdf->addInfo("Keywords", "PHP, PDF, Dompdf, Web Technologies");
9 $dompdf->addInfo("Creator", "Dompdf v3.1.5");
```

Diese Aufruf sollten nach `render()` erfolgen, denn sie greifen intern auf den Canvas zu. Dieser ist die Zeichenfläche auf dem mit Low-Level-Zeichenmethoden (ähnlich wie bei der GD Library) gezeichnet und geschrieben werden kann, falls das HTML-Rendering nicht ausreicht. Der Zugriff erfolgt über die Methode `getCanvas()`⁸⁸. Der Canvas ist auch die Schnittstelle zur darunterliegenden PDF-Bibliothek. Die Standardbibliothek „Cpdf“ kann dort über `get_cpdf()` abgefragt werden.

Fertiges Hallo Welt PDF

Das fertige Hallo Welt PDF mit serifenloser Standardschriftart (und Metadaten) ist in Abbildung 10.12 ersichtlich.

10.4.2 PDFs mit einer Template-Engine erstellen

Der Workflow, HTML-Strings direkt im PHP-Code zusammenzubauen, wird bei komplexen Dokumenten (wie Rechnungen mit Tabellen) schnell unübersichtlich. Eine mo-

⁸⁸ <https://github.com/dompdf/dompdf/wiki/Usage#canvas>

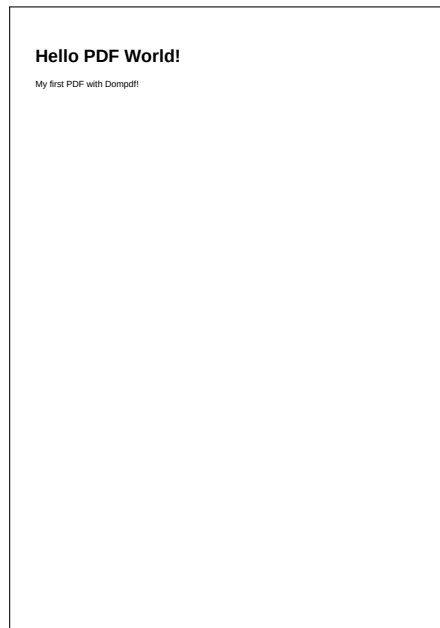


Abbildung 10.12: Das fertige Hallo Welt-PDF.

derne und professionelle Architektur trennt – wie schon in Vorlesung 6 erwähnt – aber ohnehin zwischen Logik und Präsentation. Dies trifft auch auf die PDF-Erstellung zu. Die Template-Engine *Latte* kann einfach mit Dompdf kombiniert werden.

1. In **PHP** werden die Daten zur Verfügung gestellt (als Variablen oder z. B. aus einer Datenbank).
2. **Latte** fügt die Daten in ein HTML-Template ein und generiert das fertige HTML als String.
3. **Dompdf** wandelt diesen String in ein PDF um.

Das Latte-Template

Im folgenden wird eine Übersicht über die Web Technologies-Vorlesung als Tabelle gegeben. Neben einer Überschrift wird auch noch ein Bild eingebunden. Bei diesem ist zu beachten, dass der Pfad relativ zur PHP-Datei gesetzt wird, welche die PDF-Erzeugung aufruft.

Zunächst wird eine Datei `webtechnologies.latte` im Template-Verzeichnis angelegt. Sie enthält die CSS-Formatierung im `<style>`-Tag, bindet ein lokales Bild (`images/wet2icon.png`) ein und iteriert über ein Array, um eine Tabelle aufzubauen. Dompdf unterstützt die meisten CSS 2.1-Eigenschaften zur korrekten Darstellung.

```
1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <meta charset="UTF-8">
5     <title>{$title}</title>
6     <style>
7     body {
```

```

 8     font-family: Helvetica, Arial, sans-serif;
 9     }
10     table {
11         border-collapse: collapse;
12         width: 100%;
13         margin-top: 20px;
14     }
15     th, td {
16         border: 1px solid #333;
17         padding: 8px;
18         text-align: left;
19     }
20     th {
21         background-color: #f2f2f2;
22     }
23 </style>
24 </head>
25 <body>
26     <h1>{$title}</h1>
27
28     <div class="image">
29         
30     </div>
31
32     <p>Übersicht der Vorlesungsinhalte:</p>
33
34     <table>
35         <tr>
36             <th>Einheit</th>
37             <th>Thema</th>
38         </tr>
39         <tr n:foreach="$lectures as $unit => $topic">
40             <td>{$unit}</td>
41             <td>{$topic}</td>
42         </tr>
43     </table>
44 </body>
45 </html>

```

PHP-Logik und Konvertierung

In der PHP-Datei wird nun Latte initialisiert und die Datenstruktur aufgebaut. Anstatt das Template mit `render()` direkt anzuzeigen, wird die Ausgabe über die Methode `renderToString()` in einen String geschrieben. Dieser wird dann der Methode `loadHtml()` von `Dompdf` übergeben.

Eine Besonderheit beim Verwenden von Bildern oder externen Stylesheets in `Dompdf` ist der Dateisystemzugriff. Standardmäßig blockiert `Dompdf` das Laden lokaler Dateien, um *Directory Traversal*-Angriffe zu verhindern. Um das Bild im Template korrekt rendern zu können, muss dem `Dompdf`-Objekt über die Klasse `Options` explizit ein Arbeitsverzeichnis zugewiesen werden. Der Aufruf `$options->setChroot([__DIR__]);` berechtigt den Renderer, auf das aktuelle Verzeichnis und dessen Unterverzeichnisse (wie `images`) zuzugreifen.

Nach dem Konvertieren des Strings in das PDF-Format werden über `addInfo()`

noch Metadaten gesetzt. Abschließend wird das Dokument sowohl an den Browser zur Inline-Anzeige gesendet (`stream()`) als auch dauerhaft auf dem Server gespeichert.

```

1 // Latte initialisieren
2 $latte = new Engine();
3 $latte->setLoader(new FileLoader(__DIR__ . "/templates"));
4 $latte->setCacheDirectory(__DIR__ . "/cache");
5
6 // Daten fürs Template
7 $data = [
8     "title" => "Willkommen zu Web Technologies",
9     "lectures" => [
10         "Vorlesung 1" => "Grundlagen der Kommunikation im Web",
11         "Vorlesung 2" => "PHP-Grundlagen Teil 1",
12         "Vorlesung 3" => "PHP-Grundlagen Teil 2 und Regular Expressions",
13         "Vorlesung 4" => "Objektorientiertes PHP",
14         "Vorlesung 5" => "Komponenten und Standards",
15         "Vorlesung 6" => "Templates und Routing",
16         "Vorlesung 7" => "Formularverarbeitung",
17         "Vorlesung 8" => "Sessions und Authentifizierung",
18         "Vorlesung 9" => "Datum, Zeit und Internationalisierung",
19         "Vorlesung 10" => "Medienverarbeitung",
20         "Vorlesung 11" => "REST APIs & AJAX",
21         "Vorlesung 12" => "Microframeworks und Testing",
22     ],
23 ];
24
25 // Template in einen String rendern
26 $html = $latte->renderToString("webtechnologies.latte", $data);
27
28 // Dompdf mit Optionen erzeugen
29 $options = new Options();
30 $options->setChroot(__DIR__);
31
32 $dompdf = new Dompdf($options);
33 $dompdf->loadHtml($html);
34 $dompdf->setPaper("A4", "portrait");
35 $dompdf->render();
36
37 // Metadaten hinzufügen
38 $dompdf->addInfo("Title", "Dompdf & Latte Example");
39 $dompdf->addInfo("Author", "Wolfgang Hochleitner");
40 $dompdf->addInfo("Subject", "Dompdf & Latte Demo");
41 $dompdf->addInfo("Keywords", "PHP, PDF, Dompdf, Latte, Web Technologies");
42 $dompdf->addInfo("Creator", "Dompdf v3.1.5");
43
44 // PDF anzeigen und speichern
45 $dompdf->stream("pdf-latte-webtechnologies.pdf", ["Attachment" => false]);
46 file_put_contents(__DIR__ . "/pdf-latte-webtechnologies.pdf", $dompdf->output());

```

Fertiges PDF mit Vorlesungsinhalten

Das PDF mit den Inhalten der zwölf Vorlesungen und einer Grafik, welches unter Zuhilfenahme von Latte erstellt wurde, ist in Abbildung 10.13 ersichtlich.

Willkommen zu Web Technologies



Übersicht der Vorlesungsinhalte:

Einheit	Thema
Vorlesung 1	Grundlagen der Kommunikation im Web
Vorlesung 2	PHP-Grundlagen Teil 1
Vorlesung 3	PHP-Grundlagen Teil 2 und Regular Expressions
Vorlesung 4	Objektorientiertes PHP
Vorlesung 5	Komponenten und Standards
Vorlesung 6	Templates und Routing
Vorlesung 7	Formularverarbeitung
Vorlesung 8	Sessions und Authentifizierung
Vorlesung 9	Datum, Zeit und Internationalisierung
Vorlesung 10	Medienverarbeitung
Vorlesung 11	REST APIs & AJAX
Vorlesung 12	Microframeworks und Testing

Abbildung 10.13: Das fertige PDF Dokument. Die Tabelle wurde in Latte erzeugt und gerendert, die Grafik ist aus dem Dateisystem eingebunden.

10.5 Weiterführende Literatur

Bei den in PHP verfügbaren XML-Werkzeugen hat sich in den letzten Jahren viel getan. Leider existiert nur wenig brauchbare Literatur zu dem Thema. [3] behandelt lediglich die Erweiterungen XML Parser sowie DOM und SimpleXML. XMLReader fehlt und auch auf das Thema XML schreiben wird nicht eingegangen.

Im deutschsprachigen Bereich behandelt [6] das Thema XML mit allen Erweiterungen sehr ausführlich, [5] gibt einen nicht ganz so detaillierten aber dennoch ausreichenden Überblick. JSON wird in der Literatur nur am Rande angesprochen. [5, 6] zeigen die Funktionen auf wenigen Seiten.

Da die GD Graphics Library zur PHP-Standarddistribution gehört, ist das Thema in der Literatur entsprechend gut abgedeckt. [3] widmet sich ausführlich dem Thema und wartet mit vielen Beispielen und Erklärungen auf, die auf einem aktuellen Stand sind. Auch die beiden deutschsprachigen Werke [5, 6] warten mit guten Kapiteln zum Thema GDlib auf.

Die Erzeugung von PDFs über den Workflow der HTML-zu-PDF-Konvertierung wird am besten über die Online-Dokumentation von Dompdf⁸⁹ studiert, da Printliteratur bei der Vorstellung externer Bibliotheken naturgemäß oft hinterherhinkt. [3, 6] zeigen PDF-Generierung zwar ausführlich, fokussieren sich aber auf Bibliotheken wie TCPDF, die mittlerweile als veraltet anzusehen sind.

Schließlich sei noch auf [4] verwiesen. Die offizielle Dokumentation enthält eigene Kapitel zu den nativen Themen XML, JSON und GD, in denen alle Funktionen mit Beispielen illustriert sind.

⁸⁹<https://github.com/dompdf/dompdf/wiki>

Quellenverzeichnis

- [1] *imagefilledarc*. 8. Juni 2026. URL: <https://www.php.net/manual/de/function.imagefilledarc.php> (besucht am 08.06.2026) (siehe S. 10-22).
- [2] Günter Knauf. *PHP GD Clock*. 20. Sep. 2004. URL: <https://www.codelibrary.com/snippet/655/gd-clock> (besucht am 08.06.2026) (siehe S. 10-22).
- [3] Kevin Tatroe und Peter MacIntyre. *Programming PHP. Creating Dynamic Web Pages*. 4. Aufl. Sebastopol, USA: O'Reilly, 31. März 2020 (siehe S. 10-37).
- [4] The PHP Documentation Group. *PHP-Handbuch*. 8. Juni 2026. URL: <https://www.php.net/manual/de/> (siehe S. 10-37).
- [5] Thomas Theis. *Einstieg in PHP 8 und MySQL*. 15. Aufl. Bonn, Deutschland: Rheinwerk Computing, 5. Apr. 2023 (siehe S. 10-37).
- [6] Christian Wenz und Tobias Hauser. *PHP 8 und MySQL. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Mai 2024 (siehe S. 10-37).